SHIELDAPI:

Readme.md

ShieldAPI: Intelligent API Security Gateway 🛡️ 🚀

ShieldAPI is a production-grade security wrapper for Spring Boot applications. It provides an intelligent, layered defense system designed to monitor, detect, and automatically mitigate malicious API usage in real-time.

✨ Key Features

- **Layered Security Pipeline**: Multi-stage filtering process including IP reputation, rate limiting, and JWT authentication.
- **Intelligent Threat Detection**: Asynchronous behavioral analysis that tracks suspicious patterns and calculates real-time threat scores.
- **Automated Mitigation**: Dynamic IP blacklisting based on configurable risk thresholds.
- **Real-time Event Streaming**: Seamless integration with Apache Kafka for exporting security events to SIEM/SOAR platforms.
- **Administrative Analytics**: Secure (ROLE_ADMIN) dashboards for monitoring threat summaries, top attackers, and full audit trails.
- **Non-blocking Observability**: Async audit logging and Spring Actuator integration for high-performance monitoring.

🛠️ Technology Stack

- **Framework**: Spring Boot 3.4
- **Security**: Spring Security 6 (Stateless JWT)
- **Throttling**: Bucket4j (token bucket algorithm)
- **Messaging**: Spring Kafka
- **Persistence**: Spring Data JPA / H2 Database (In-Memory)
- **Monitoring**: Spring Boot Actuator / Prometheus

📚 Documentation

Deep-dive documentation is available in the [`docs/`](./docs) directory:

- [**Architecture Overview**](./docs/architecture.md): High-level system design and request flow logic.
- [**Security Flow**](./docs/security-flow.md): Detailed breakdown of the multi-layered security processing pipeline.
- [**Threat Model**](./docs/threat-model.md): Detailed analysis of mitigated threats and defensive strategies.
- [**System Components**](./docs/system-components.md): Inventory of filters, services, and modules.

🚀 Getting Started

Prerequisites

- Java 17+
- Maven 3.x
- Apache Kafka (for event streaming)

Running the Application

```bash
./mvnw clean spring-boot:run
```

The application will be available at `http://localhost:8080`.

Default Credentials (for testing)

- **H2 Console**: `http://localhost:8080/h2-console` (JDBC URL: `jdbc:h2:mem:shieldapi_db`)

🕵️ Monitoring & Analytics

Administrators can access the following endpoints:

- `GET /api/v1/admin/security/threat-summary`: Aggregated threat metrics.
- `GET /api/v1/admin/security/top-attacking-ips`: Most active malicious sources.
- `GET /api/v1/admin/security/audit-logs`: Full system audit trail.

Architecture.md

ShieldAPI Architecture Overview

ShieldAPI is an intelligent security gateway designed to protect backend services through a layered defense-in-depth approach. It integrates real-time monitoring, automated threat mitigation, and event streaming.

High-Level Architecture

The system follows a sequential processing pipeline where every request is validated against multiple security layers before reaching the application logic.

Request Flow Diagram

```mermaid
graph TD
  Client[Client Request] --> AF[IpReputationFilter]
  AF --> RL[RateLimitFilter]
  RL --> JWT[JwtAuthenticationFilter]
  JWT --> CTRL[Controllers]
  CTRL --> SVC[Services]
  SVC --> DB[(H2 Database)]

  subgraph "Intelligent Security Layer"
    AF
    RL
    JWT
  end
```

```
    subgraph "Background Processing"
        TI[ThreatIntelligenceService]
        KB[Kafka Event Stream]
        AL[Audit Logging]
    end

    AF -.-> TI
    RL -.-> TI
    JWT -.-> TI
    TI -.-> KB
    CTRL -.-> AL
```

Core Architectural Layers

1. Security Filter Chain

The primary entry point for all requests. Built on Spring Security, it enforces:

- **IP Reputation**: Immediate rejection of blacklisted IP addresses.
- **Adaptive Rate Limiting**: Request throttling based on client IP to prevent DoS attacks.
- **Identity & Access**: Stateless JWT validation to ensure only authorized users access protected resources.

2. Threat Detection Pipeline

An asynchronous engine that monitors for suspicious patterns:

- **Event Correlation**: Linking multiple failures (e.g., auth failures + rate limit hits) to a single source.
- **Dynamic Scoring**: Incrementing threat scores based on action severity.
- **Automated Mitigation**: Triggering IP blocks when threat thresholds are exceeded.

3. Analytics & Monitoring

Real-time visibility into system health:

- **Security Analytics**: Dedicated endpoints for administrators to view threat summaries and attack patterns.

- **Audit Trails**: Detailed request/response logging for compliance and forensic analysis.

- **Kafka Streaming**: Real-time event export to external SIEM/SOAR platforms.

4. Persistence Layer

Uses Spring Data JPA with an H2 database for storing:

- Blocked IP lists

- Historical threat events

- System audit logs

- User and role configurations

Security-flow.md

Security Processing Flow

This document details how ShieldAPI processes every incoming request to ensure maximum security and minimum latency.

The Security Pipeline

ShieldAPI implements a "Fail Fast" security model. Requests are evaluated in stages; if any stage fails, the request is rejected immediately, saving downstream resources.

Step 1: IP Reputation Check (`IpReputationFilter`)

- **Action**: Intercepts the request and checks the client's IP against the `blocked_ips` table.

- **Outcome**: If blocked, returns `403 Forbidden`. If clean, proceeds to the next layer.

Step 2: Rate Limiting (`RateLimitFilter`)

- **Action**: Uses **Bucket4j** to track requests per IP address.

- **Outcome**: If the bucket is empty (limit exceeded), it records a `RATE_LIMIT_ABUSE` threat and returns `429 Too Many Requests`.

Step 3: Authentication (`JwtAuthenticationFilter`)

- **Action**: Extracts the `Authorization` header and validates the JWT.

- **Outcome**:

  - **Valid**: Sets the `SecurityContext` with user details and roles.

  - **Missing/Invalid**: Access is denied (unless the endpoint is public, e.g., `/auth/**`).

Step 4: Authorization

- **Action**: Spring Security checks the authenticated user's roles against the endpoint's requirements (e.g., `ROLE_ADMIN` for analytics).

- **Outcome**: Returns `403 Forbidden` if roles are insufficient.

Step 5: Audit & Activity Tracking (`ApiAuditInterceptor`)

- **Action**:

  - `preHandle`: Records the request start time.

  - `afterCompletion`: Captures URI, method, status code, response time, and user ID.

- **Outcome**: Persists an `ApiAuditLog` asynchronously to minimize latency.

Threat Detection Logic

While the request flows through the filters, the **Threat Detection Engine** works in parallel:

1. **Detection**: If a filter (or the Auth Failure Handler) detects a violation, it calls `ThreatIntelligenceService.recordThreat()`.

2. **Scoring**: The `ThreatScoreService` updates the internal "Maliciousness Score" for that IP.

3. **Blacklisting**: If the score reaches a critical threshold, the `IpReputationService` automatically blacklists the IP for 7 days.

4. **Streaming**: A detailed `SecurityEvent` is published to the `shieldapi-security-events` Kafka topic.

Threat-model.md

ShieldAPI Threat Model

ShieldAPI is designed to mitigate common web-based attack vectors through intelligent detection and automated response.

Mitigated Threats

1. Brute Force Attacks

- **Description**: Repeated attempts to guess credentials on authentication endpoints.
- **Detection**: `CustomAuthenticationFailureHandler` triggers after every failed login attempt.
- **Mitigation**:
  - Each failure increases the IP's threat score.
  - High-frequency failures trigger an automatic IP block.

2. Excessive Request Flooding (DoS)

- **Description**: Flooding the API with requests to exhaust resources and cause denial of service.
- **Detection**: `RateLimitFilter` monitors the request rate per IP.
- **Mitigation**:
  - Immediate `429 Too Many Requests` response.
  - Repeated violations lead to longer-term blacklisting.

3. Suspicious Endpoint Scanning

- **Description**: Probing for sensitive endpoints or known vulnerabilities (e.g., trying to access admin paths without authority).
- **Detection**: Security filters and controllers monitor for repeated `401` or `403` responses.
- **Mitigation**: Automatically records a `THREAT_DETECTED` event with the targeted endpoint.

4. Credential Stuffing

- **Description**: Using lists of leaked credentials to gain unauthorized access.
- **Detection**: Monitored via the `AUTH_FAILURE` event stream in Kafka.
- **Mitigation**: Real-time alerts to security teams via the event stream for coordinated response.

Defensive Strategies

| Defense Layer | Threat Category | Action Taken |

| :--- | :--- | :--- |

| **IP Reputation** | Known Malicious Sources | Direct Block (Hard Drop) |

| **Rate Limiter** | Bot Activity / Scraping | Request Throttling |

| **JWT Validator** | Unauthorized Access | Identity Verification |

| **Threat Scorer** | Coordinated Attacks | Behavioral Analysis & Dynamic Blocking |

Event Severity Levels

- **LOW**: Minor rate limit hits or single malformed requests.
- **MEDIUM**: Multiple auth failures or repeated scanning.
- **HIGH**: Threshold reached – IP blacklisted.
- **CRITICAL**: Immediate system-wide alerts (via Kafka).

System-components.md

System Components & Responsibilities

ShieldAPI is built on a modular architecture where each component has a clearly defined responsibility. This document provides a high-level overview of these components.

1. Security Filters

| Component | Responsibility |

| :--- | :--- |

| `IpReputationFilter` | Immediate rejection of requests from blacklisted IP addresses. |

| `RateLimitFilter` | Enforces client-specific request quotas using **Bucket4j**. |

| `JwtAuthenticationFilter` | Extracts and validates JWT tokens; manages the **SecurityContext**. |

2. Security Services

| Component | Responsibility |

| :--- | :--- |

| `ThreatIntelligenceService` | Orchestrates threat detection, recording, and scoring. |

| `ThreatScoreService` | Calculates IP-based threat scores using weighted risk categories. |

| `IpReputationService` | Logic for blacklisting and unblocking IP addresses. |

| `RateLimitService` | Manages and resolves request buckets for various client IPs. |

3. Monitoring & Analytics

| Component | Responsibility |

| :--- | :--- |

| `SecurityAnalyticsController` | Provides secure endpoints for admin monitoring. |

| `SecurityAnalyticsService` | Aggregates threat data and audit logs for the analytics API. |

| `ApiAuditInterceptor` | Intercepts requests/responses to log audit data synchronously and then persists it asynchronously. |

| `SecurityEventPublisher` | Publishes structured security events to **Apache Kafka**. |

| `SecurityMetricsService` | Tracks in-memory counters for real-time dashboarding. |

4. Repositories & Entities

- **`ThreatEvent`**: Entity for logging specific security violations.
- **`BlockedIpAddress`**: Entity for persistent blacklisting data.
- **`ApiAuditLog`**: Entity for general request/response auditing.
- **`User`**: Entity for application identity and role-based access control.

5. Interaction Examples

Threat Mitigation Flow

1. `CustomAuthenticationFailureHandler` detects a brute force attempt.
2. It notifies `ThreatIntelligenceService`.
3. `ThreatIntelligenceService` updates the `ThreatEventRepository` and calls `ThreatScoreService`.
4. If the score is too high, `IpReputationService` blacklists the IP.
5. `SecurityEventPublisher` streams the event to Kafka for external monitoring.